WEEK 2 · LESSON 8 OF 24 · HANDOUT

Random Forests: The Wisdom of Many Trees

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=J4Wdy0Wc_xQ — open the lesson online to watch inside the course.

🎓 **Go deeper:** SciPy 2018 tutorial: Machine Learning with scikit-learn, Part 2 (scikit-learn core developers) — https://www.youtube.com/watch?v=gK43gtGh49o

Your goal in this lesson: Explain bagging and random feature selection, train a random forest, and use out-of-bag scores and feature importance.

One tree is shaky, many trees are steady

Ask one person to guess the number of beans in a jar and the guess may be far off. Ask 300 people and take the average: the answer is usually close. Errors of different people **cancel out**. A **random forest** uses the same idea: train many different trees and let them **vote**. A group of models working together is called an **ensemble**.

Two sources of randomness

For the trees to make **different** mistakes, each one is trained differently:

1. **Bootstrap samples (bagging).** Each tree gets its own random sample of the training rows, drawn **with replacement** — some rows appear twice, about a third are left out.
2. **Random features.** At every split, a tree may only choose from a random subset of the features (by default √(number of features) for classification).

The prediction is the **majority vote** of all trees (or the **average** for regression).

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(11)
n = 2000
income = rng.lognormal(12.2, 0.45, n).round(-3)
loan = rng.uniform(20_000, 300_000, n).round(-3)
months = rng.integers(1, 61, n)
late = rng.poisson(0.6, n)
salary = rng.random(n) < 0.55
z = -3.2 + 2.5 * (loan / income) - 0.03 * months + 0.8 * late - 0.9 * salary
default = (rng.random(n) < 1 / (1 + np.exp(-z))).astype(int)
loans = pd.DataFrame({"income_rwf": income, "loan_rwf": loan, "months_customer": months,
                      "late_payments": late, "salaried": salary.astype(int),
                      "defaulted": default})
print(loans.shape, "default rate:", round(loans["defaulted"].mean(), 3))
loans.head()
```

```
from sklearn.model_selection import train_test_split

X = loans.drop(columns="defaulted")
y = loans["defaulted"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)
```

Train a forest

```
from sklearn.ensemble import RandomForestClassifier

forest = RandomForestClassifier(n_estimators=300, random_state=42, n_jobs=-1)
forest.fit(X_train, y_train)
print("Train:", round(forest.score(X_train, y_train), 3))
print("Test: ", round(forest.score(X_test, y_test), 3))
```

- `n_estimators` — the number of trees. More trees are never worse, just slower; 100–500 is typical.
- `n_jobs=-1` — use all the computer's processor cores.

Each tree is grown deep, so the forest scores almost **100%** on training data, but averaging makes it far more stable than one deep tree: about **0.83** on the test set instead of **0.76**.

Make the trees less extreme

```
forest5 = RandomForestClassifier(n_estimators=300, min_samples_leaf=5,
                                 random_state=42, n_jobs=-1).fit(X_train, y_train)
print("Train:", round(forest5.score(X_train, y_train), 3),
      "Test:", round(forest5.score(X_test, y_test), 3))
```

Asking for at least 5 borrowers per leaf narrows the train–test gap and nudges the test score up to about **0.84**.

A free test score: out-of-bag

Each tree never saw about a third of the rows (the ones left out of its bootstrap sample). The forest can test each row using only the trees that did **not** train on it. This **out-of-bag (OOB) score** is an honest estimate without touching the test set:

```
oob = RandomForestClassifier(n_estimators=300, oob_score=True,
                             random_state=42, n_jobs=-1).fit(X_train, y_train)
print("OOB score:", round(oob.oob_score_, 3))
```

It comes out at about **0.85**, close to the test score.

What did the forest use?

```
imp = pd.Series(oob.feature_importances_, index=X.columns).sort_values()
print(imp.round(2))
imp.plot(kind="barh", title="Random forest feature importance")
```

Income and loan size are the most important, then months as a customer. Unlike the single small tree, the forest uses **every** feature a little.

Note: impurity-based importance can favour features with many different values.
`sklearn.inspection.permutation_importance` — which shuffles one feature at a time and measures how

much the score drops — is a more trustworthy check.

Random forests for numbers

```
from sklearn.ensemble import RandomForestRegressor
# RandomForestRegressor(n_estimators=300).fit(X_homes, y_rent) — the average of 300 trees
```

When to use a random forest

Good at	Less good at
Strong results with little tuning	Explaining a single decision simply
No scaling, handles curved patterns and interactions	Very large data (memory and speed)
Robust to noisy features	Predicting beyond the range of the training data

Random forests are one of the best **first models** to try on any table of data.

Key terms

- **Ensemble** — many models combined into one.
- **Bagging (bootstrap aggregating)** — train each model on a random sample drawn with replacement, then combine.
- **Random forest** — bagged decision trees that also pick features at random.
- **Out-of-bag score** — accuracy measured on rows each tree did not see.
- **Permutation importance** — score drop when a feature is shuffled.

Remember this: A random forest averages many different deep trees, which keeps their power but cancels most of their overfitting.

Name: _____ Date: _____

Activity

Task 1

Train forests with 10, 50, 100 and 500 trees. Plot the test accuracy. When does adding trees stop helping?

Task 2

Set `max_features=None` (every split may use every feature). Does the forest get better or worse? Why?

Task 3

Run `permutation_importance(oob, X_test, y_test, n_repeats=10, random_state=0)` and compare with the impurity importance.

Task 4

Train a `RandomForestRegressor` on the homes data and compare its MAE with linear regression from Lesson 3.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What is an ensemble?
   - A. A type of scaler
   - B. Many models combined into one prediction
   - C. One very deep tree
   - D. A dataset
2. What is a bootstrap sample?
   - A. The features only
   - B. The first 100 rows
   - C. A random sample of rows drawn with replacement
   - D. The test set
3. Why does a random forest also choose features at random at each split?
   - A. To make the trees different so their mistakes cancel
   - B. To scale the data
   - C. To save memory
   - D. To remove the target
4. How does a random forest classifier make its final prediction?
   - A. It uses the first tree
   - B. The deepest tree decides
   - C. Majority vote of all trees
   - D. It picks a random tree
5. What is the out-of-bag score?
   - A. The test accuracy
   - B. The number of trees
   - C. Accuracy on rows each tree did not train on
   - D. The training accuracy
6. What usually happens when you add more trees?
   - A. The forest overfits much more
   - B. Results stabilise; it is never worse, only slower
   - C. Scaling becomes necessary
   - D. Accuracy drops to zero

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.